AWS Generative AI

Thursday, March 19, 2026 10:00 PM

Mock Test: AWS Certified Generative AI Developer – Professional

1.A company is developing an AI assistant that processes customer data by using Amazon Bedrock. The AI assistant has multiple guardrails. The guardrails include prompt injection detection, sensitive information filtering, and denied topic blocking. When a customer query is blocked, a GenAI developer needs a detailed analysis of which specific guardrail rule was invoked and why the content was flagged. Then, the GenAI developer must fine-tune guardrail configurations and distinguish between legitimate customer queries and actual security threats. Which configuration provides the MOST detailed analysis of guardrail decision-making for content filtering?

1. Enable Amazon Bedrock model evaluation with automated evaluation jobs that include guardrail assessment metrics. Configure the evaluation framework to test prompt injection resistance by using company-specific test cases. Use the evaluation dashboard to analyze which guardrail policies are most effective at blocking malicious content while preserving legitimate queries.
2. Enable Amazon Bedrock model invocation logging to capture full request and response data. Configure Amazon CloudWatch alarms on InvocationIntervened metrics filtered by GuardrailContentSource dimensions. Analyze patterns by using CloudWatch insights queries to identify which content source triggered interventions.
3. Configure guardrail tracing with '{"trace":"enabled"}' in guardrailConfig. Monitor InvocationIntervened metrics filtered by the GuardrailContentSource dimension to identify weather input prompts or output responses triggered interventions.
4. Configure guardrail tracing with '{"trace":"enabled"}' in guardrailConfig. MonitorInvocationIntervened metrics filtered by the GuardrailPolicyType dimensions ContentPolicy, TopicPolicy and SensitiveInformationPolicy.

2.A financial services company wants to develop a mobile app that will help users with account inquiries and general account information. The company has a large amount of email exchange data between customers and support staff to use as source material. The data is stored in an Amazon S3 bucket and contains personally identifiable information (PII) that should not appear in search results. Which solution will meet these requirements?

1. Use Amazon Kendra to enable enterprise search of the email data that is stored in Amazon S3. Integrate Amazon Kendra with an Amazon Bedrock FM. Use a system prompt to identify and remove PII during query processing.
2. Use Amazon Comprehend to detect and redact PII from the email data that is stored in Amazon S3. Integrate Amazon Comprehend with Amazon Kendra to enable enterprise search of the processed data.
3. Use Amazon Textract to extract text from the email data. Use Amazon Macie to scan for PII in Amazon S3. Integrate Amazon Textract and Amazon S3 with Amazon Kendra to enable enterprise search of the processed data.
4. Use Amazon Comprehend to detect and redact PII from the email data that is stored in Amazon S3. Integrate Amazon Comprehend with Amazon DocumentDB to enable database queries for enterprise search.

3.A financial services company is developing a research agent that processes complex financial data queries. The company must deploy existing Python agent code to Amazon Bedrock AgentCore Runtime. The company wants to reduce infrastructure management overhead and operational complexity. The agent must be able to handle quick data lookups that require sub-second responses. The agent must be able to handle comprehensive research report generation. For example, streaming responses over several minutes. The solution must automatically manage HTTP server configuration, endpoint routing, and health monitoring. Which deployment approaches will meet these requirements with MINIMAL operational overhead? (Select TWO.)

1. Implement a FastAPI server with a configuration of /invocations and /ping endpoints and container orchestration.
2. Implement the AgentCore SDK with the @app.entrypoint decorator to automatically handle server setup and endpoint management.
3. Deploy the agent on Amazon ECS on AWS Fargate by using a custom container image that runs the AgentCore SDK application.
4. Deploy the agent on Amazon SageMaker AI real-time endpoints by using a custom inference container.

5. Deploy the agent by using the AgentCore starter toolkit for automated packaging, containerization, and deployment workflows.

4.A GenAI developer is implementing a solution to **create images from text descriptions**. The GenAI developer successfully tested a pre-trained Hugging Face model by using Amazon SageMaker JumpStart. Now, the GenAI developer needs to deploy the model so that users can generate images **on demand**. The solution must **use GPUs for inferen**ce. The solution must be able to handle **text datasets up to 50 MB with image descriptions**. The solution requires **responses within 15 minutes.** Which deployment strategy will meet these requirements?

1. Deploy a SageMaker Asynchronous Inference endpoint that uses an accelerated computing SageMaker AI instance type. Create an AWS Lambda function for on-demand invocation of the SageMaker AI endpoint to manage image generation.
2. Deploy a SageMaker Serverless Inference endpoint that uses a general purpose SageMaker AI instance type. Create an AWS Lambda function for on-demand invocation of the SageMaker AI endpoint to manage image generation.
3. Deploy a SageMaker Real-Time Inference endpoint that uses an accelerated computing SageMaker AI instance type. Create an AWS Lambda function for on- demand invocation of the SageMaker AI endpoint to manage image generation.
4. Deploy a SageMaker AI batch transform job that uses an accelerated computing SageMaker AI instance type to manage image generation. Create an AWS Lambda function to start the batch transform job.

5.A company is developing a RAG application by using Amazon Bedrock. The application processes customer support documents. Initially, the application retrieves many relevant documents. However, users report that the most relevant information often appears lower in the results. The company wants to improve the relevance ranking of retrieved results to ensure that the most useful information appears first. Which combination of steps will improve the relevance of retrieved results with MINIMAL operational overhead? (Select TWO.)

1. Configure Amazon OpenSearch Serverless with the Amazon Bedrock Knowledge Bases plugin. Use OpenSearch's Learning to Rank feature for relevance scoring. Integrate relevance scoring with Knowledge Bases for result reranking.
2. Create an Amazon Aurora PostgreSQL database with the pgvector extension to store document embeddings. Create a similarity scoring algorithm that combines vector distances with document metadata to rank results.
3. Use Amazon Bedrock reranker models with Amazon OpenSearch Service to reorder retrieved results based on semantic relevance to the query.
4. Use Amazon SageMaker JumpStart FMs with Amazon Kendra intelligent Ranking to create custom relevancy scoring algorithms.
5. Use Knowledge Bases with hybrid search capabilities and Amazon OpenSearch Serverless to combine vector embeddings with keyword matching.

6.A company wants to create an application to **analyze fashion trends**. The application must analyze videos and photos from public fashion shows to understand style elements and trends. The solution must **store the extracted information** and **provide a dashboard that summarizes the fashion trend**s. Which solution will meet these requirements with the L**EAST operational overhead?**

1. Use Amazon EventBridge to trigger AWS Lambda functions that use Amazon QuickSight Q to analyze videos and photos from fashion shows. Store analysis results in Amazon S3. Deploy a QuickSight dashboard with ML-powered trend analysis.
2. Use AWS Step Functions to process videos and photos from fashion shows by using Amazon Bedrock multimodel FMs. Store analysis results in Amazon S3. Use an Amazon QuickSight dashboard to visualize trends in the data.
3. Use Amazon Rekognition Custom Labels to train a custom model for fashion trend analysis. Store results in Amazon DynamoDB. Create a dashboard using Amazon Managed Grafana with custom plugins for fashion trend analysis.
4. Use an Anthropic Claude model in Amazon Bedrock to analyze text descriptions from fashion show videos and photos. Use Stable diffusion for image analysis. Store results in Amazon OpenSearch Service. Create a dashboard using Amazon Managed Grafana with OpenSearch visualizations.

7.A cross-functional team is developing a generative AI (GenAI) application by using AWS services. The team needs to **optimize developer productivity** and **enforce consistent integration patterns**. The team needs to **automate performance tuning and accelerate AI testing** across multiple business units. The team wants to use **Amazon Q Develope**r. The team must accelerate development workflows and maintain application quality. Which combination of steps will meet these requirements? (Select TWO.)

1. Use Amazon Q Developer to analyze code for security best practices and suggest compliance improvements. Implement a mandatory review process where all code changes must be manually approved by security teams before integration.
2. Use Amazon Q Developer to retrospectively analyze and document common integration patterns that are found across different business units code bases.
3. Configure Amazon Q Developer to automatically generate and refactor integration code snippets, provide targeted API usage guidance, and suggest performance optimizations for AI components. Apply the changes across the modular code base.
4. Incorporate Amazon Q Developer to resolve coding issues that are identified during merge requests. Reserve most refactoring and optimization tasks for periodic manual review cycles.
5. Integrate the Amazon Q Developer automated unit and integration test generation features into the team's CI/CD pipelines.

8.A company uses an **AI assistant to answer customer questions based on internal company documents**. The company **wants to include new documents** in the a**ssistant's response**s as soon as possible. The company wants to **exclude deleted documents from the AI assistant's responses** as soon as possible. The **documents are stored in Amazon S3**. The AI assistant uses Amazon Bedrock Knowledge Bases. A**mazon S3 is the data source of the vector stor**e that the company uses for **RAG**. A GenAI developer must create a s**calable, event-driven, and resilient solutio**n. Which solution will meet these requirements?

1. Configure Amazon EventBridge schedular to schedule a rule that runs every 5 minutes and invokes an AWS Lambda function. Configure the Lambda function to track changes in Amazon S3 and invoke IngestKnowledgeBaseDocuments for new objects and DeleteKnowledgeBaseDocuments for deleted objects.
2. Configure S3 Event Notifications to invoke an AWS Lambda function on object- created and object-deleted events. Configure the Lambda function to invoke IngestKnowledgeBaseDocuments for new objects and DeleteKnowledgeBaseDocuments for deleted objects.
3. Configure S3 Event Notifications to send object-created and object-deleted events to an Amazon SQS queue. Create an AWS Lambda function to poll the queue and invoke IngestKnowledgeBaseDocuments for new objects and DeleteKnowledgeBaseDocuments for deleted objects.
4. Configure Amazon EventBridge schedular to schedule a rule that runs every 5 minutes and invokes an AWS Lambda function. Configure the Lambda function to sync the documents in Amazon S3 with the knowledge base by invoking StartIngestionJob.

9.A company is implementing AI **governance policies.** The policies require all **FM interactions to be secured with guardrail**s. The company configures Amazon Bedrock guardrails. The company must ensure that all I**nvokeModel and Converse API calls to FMs apply the guardrai**ls. Which solution will enforce guardrail compliance for the API calls in the MOST operationally efficient way?

1. Configure IAM policies for the InvokeModel and Converse API calls with both bedrock:GuardrailIdentifier and bedrock:PromptRouterArn condition keys. Apply the policies to all IAM roles. Require prompt router validation before allowing access to Amazon Bedrock FMs.
2. Create an AWS Lambda function that validates and enforces guardrails before proxying requests to Amazon Bedrock. Use the Lambda function as the exclusive endpoint for all FM interactions.
3. Store guardrail identifiers in AWS Systems Manager Parameter Store. Create an AWS Lambda function that retrieves the guardrail identifier from Parameter store each time before making calls to Amazon Bedrock FMs.
4. Configure IAM policies for the InvokeModel and Converse API calls with the bedrock: GuardrailIdentifier condition key. Apply the policies to all IAM roles that access the Amazon Bedrock FMs.

10.A news media company wants to develop a content conformance tool that a**utomatically reviews and adjusts articles to ensure compliance with a style guid**e. Journalists need a **web-based article edito**r that provides r**eal-time analysis of content upon request**. When journalists click an "**analyze**" button, the s**ystem should immediately begin providing suggested revisions** through the editor interface. **Articles are tagged with content categories in the metadat**a. Examples of categories include **news, sports, and editorial**. The company wants to use an **Amazon Bedrock FM to analyze content** and provide **immediate feedbac**k through the web-based article editor interface. Which architecture will meet these requirements with the **LEAST operational overhead?**

1. Implement an Amazon SQS queue for article ingestion. Create AWS Step Functions workflows to process content. Use AWS Lambda functions to determine the content category from metadata and invoke appropriate Amazon Bedrock models with style guide prompts. Store results in Amazon DynamoDB. Use an Amazon API Gateway WebSocket API for real-time streaming of suggestions to the journalists.

2. Deploy an Amazon API Gateway WebSocket API linked to an AWS Lambda function. Configure the function to read the content category from metadata and route content to the appropriate Amazon Bedrock model based on the category tag. Configure the function to use Amazon Bedrock Prompt Management to enforce style guide rules. Use the Amazon Bedrock streaming API to return suggestions in real-time.
3. Create an Amazon API Gateway REST API with AWS Lambda function URLs to enable response streaming. Configure the lambda function to process articles and stream suggestions using chunked transfer encoding.
4. Configure an Application Load Balancer with Amazon ECS tasks that run custom containers. Implement content category routing logic and style guide checking within the containers. Use Amazon Bedrock with streaming support to generate suggestions. Use WebSocket connections to stream results to the journalist in real time.

11.An education company built a content generation system on Amazon Bedrock. The system generates practice questions to quiz end users on a topic to test their knowledge. The system consumes a mix of curated data and scraped data in the topic domain. Reviewers must approve of the generated question-response sets before end users can access the sets. The company wants to improve the system by adding source lineage for the reviewers to verify the credibility of the content. Which combination of steps will meet these requirements with the LEAST operational overhead? (Select TWO.)

1. Enable Amazon Bedrock invocation logging and correlate the logs with the data source.
2. Tag FM outputs with metadata from the data source.
3. Register the curated and scraped input datasets with AWS Glue Data Catalog.
4. Use Amazon SageMaker Clarify to explain model predictions.
5. Use AWS CloudTrail to log reviewer feedback actions.

12. A company runs a question-answering application. The application uses an Amazon Bedrock knowledge base that ingests documents from multiple Amazon S3 buckets. The company needs to monitor the data ingestion process to identify and troubleshoot any issues with document processing. Which solution will meet these requirements to monitor knowledge base operations?

1. Configure knowledge base logging with Amazon CloudWatch Logs as the destination. Use CloudWatch Logs insights to query the failed document processing.
2. Enable Amazon CloudWatch Application Signals to automatically detect and alert on knowledge base performance issues.
3. Enable AWS CloudTrail to track all API calls that relate to knowledge base operations and document ingestion activities.
4. Implement Amazon Bedrock model invocation logging to capture detailed metrics about document processing and embedding generation.

13.A financial services company operates RAG for an application that answers user questions by using internal market analysis reports. The application uses Amazon Bedrock for the embedding model. The application uses an Amazon OpenSearch Service cluster as the vector store. An AWS Lambda function performs the embedding and search logic. After a recent code update to the Lambda function, the application starts returning generic responses. For example, the application returns “no relevant information found” even for questions that previously returned accurate answers. Amazon CloudWatch Logs shows no errors. AWS X-Ray confirms successful FM invocation. The OpenSearch Service cluster is healthy. Query latency remains normal. What is the cause of this issue?

1. The document embeddings in OpenSearch service were deleted during the application update and have not been re-indexed.
2. The Lambda function’s IAM role is missing the permission for bedrock InvokeModel.
3. The Amazon Bedrock FM temperature parameter was increased.
4. The updated Lambda function uses a different version of the embedding model.

14.A financial services company needs to use Amazon Bedrock to create an AI assistant that will help customer support representatives across multiple business units. A GenAI developer must ensure that prompt templates are properly governed through approval workflows. Additionally, the company requires comprehensive logging of all model invocations with a 7-year retention period for regulatory compliance. Which combination of steps will meet these requirements with MINIMAL operational overhead? (Select TWO.)

1. Use Amazon Bedrock Prompt Management with multi-stage approval workflows. Use IAM policies that require multi-party authorization.
2. Store prompt templates in Amazon DynamoDB tables with composite keys partitioned by business units. Implement IAM policies that grant role-based access to business units for template approval. Use DynamoDB item-level permissions to control prompt templates modifications and approvals.

3. Set up Amazon EventBridge rules to capture Amazon Bedrock model invocation events. Route events to Amazon CloudWatch Logs groups that are organized by business unit. Export the logs to Amazon S3. Enable S3 Object Lock with compliance retention mode set to 7 years.
4. Enable AWS CloudTrail data events for Amazon Bedrock APIs. Deliver the logs to CloudTrail Lake with a 7-year retention setting. Tag each event with the business unit ID. Run CloudTrail Lake queries to monitor prompt activity.
5. Enable Amazon Bedrock model invocation logging with Amazon S3 as the destination. Enable S3 Object Lock with compliance retention mode set to 7 years. Create separate prefixes for each business unit.

15.A company needs secure authentication for a third-party application that uses Amazon Bedrock. The solution must integrate with the company's existing identity provider (IdP). The solution must maintain comprehensive audit logs of authentication and API calls. The solution must eliminate long-lived credentials and provide temporary access to Amazon Bedrock. Which solutions will meet these requirements? (Select TWO.)

1. Implement an OpenID Connect (OIDC) integration with Amazon Cognito. Configure the integration to authenticate users through the Idpand exchange tokens for temporary credentials. Configure the integration to allow the application to access Amazon Bedrock.
2. Configure an Amazon API Gateway Lambda authorizer. Configure the authorizer to validate credentials against the company's LDAP server and then issue signed JSON Web Tokens (JWTs) for Amazon Bedrock access.
3. Create IAM users for each employee that needs access to the application. Assign permissions through IAM policies. Implement credential rotation by using AWS Secrets Manager.
4. Create an IAM role and configure federation by using AWS STS AssumeRole API calls. Store the application's IAM user credentials in the application configuration.
5. Deploy AWS IAM Identity Center with SAML federation to the IdP. Configure custom permission sets that grant access to Amazon Bedrock.

16.A company is building a diagnostic imaging application. The application needs to perform similarity searches across 50 million images to assist with diagnosing and treating patients. The application must process new images daily. The application will perform similarity searches infrequently when users need to find similar cases for reference. The company wants a cost-effective solution that provides responsive search performance without requiring infrastructure management. Which solution will meet these requirements MOST cost-effectively?

1. Store image vectors in Amazon OpenSearch Serverless. Use vector search capabilities for similarity searches.
2. Use Amazon DynamoDB to store image vectors. Implement custom similarity search logic by using AWS Lambda functions.
3. Create an Amazon S3 vector bucket with vector indexes to store image embeddings and perform similarity searches.
4. Store image vectors in Amazon RDS for PostgreSQL. Use the pgvector extension to perform similarity searches using index vector embeddings.

17.A GenAI developer deployed a fine-tuned LLM to an Amazon SageMaker AI endpoint. The GenAI developer used the default serving configuration for continuous batching with the AMI including the Deep Java Library (DJL). The model is being served on GPU- based Amazon EC2 instances, each with 8 GPUs. As the model scales to production, the GenAI developer discovers that many instances are needed to meet traffic demands. The GenAI developer wants to avoid increased costs from the overutilization. The GenAI developer analyzes logs. The GenAI developer discovers that the maximum I/O sequence length in real requests is 10 times smaller than what the model was originally configured to handle. Additionally, the current concurrency for each instance is low. Profiling shows that the model's weights and activations can fit entirely within 4 GPUs. Which combination of steps can the GenAI developer take to improve resource utilization? (Select TWO.)

1. Increase the number of SageMaker AI instances and spread requests more evenly to reduce the load for each instance.
2. Reduce the model's maximum sequence length to provide a higher rolling batch size for each GPU.
3. Enable speculative decoding to reduce response latency for each request.
4. Use tensor parallelism with a degree of 4 to deploy two model replicas for each instance.
5. Split the model across all 8 GPUs by using a tensor parallelism degree of 8 to improve memory efficiency.

18.A GenAI developer is building a virtual assistant application by using an Anthropic Claude model on Amazon Bedrock. The application sends user queries and expects conversational responses. The GenAI developer wants to configure the application to stop generating output after a specific phrase is generated in the response. Which solution will meet these requirements?

1. Add the trigger phrase "stop at this phrase" in the user prompt.

2. Use the stop sequences parameter in the inference call to specify a trigger phrase.
3. Use the top-k parameter to control the diversity of tokens in the model's output.
4. Use the temperature parameter in the inference call to control the likelihood of the phrase appearing.

19.An ecommerce company has an application that uses Amazon Bedrock to generate product descriptions and recommendations. Currently, the application resides in a single AWS Region. When invoking a model in Amazon Bedrock during peak periods, the application receives an error. The error message says, "Too many requests, please wait before trying again." The company must increase the throughput for invocations during peak periods without introducing additional operational overhead. The company must maintain compatibility with the existing Amazon Bedrock API. The company must use the same FM. Which solution will meet these requirements in the MOST cost-effective way?

1. Create an AWS Lambda function to invoke the model in Amazon Bedrock with the original region as the default. Configure the Lambda function to fall back to Amazon Bedrock in a secondary region.
2. Use cross-region inference to distribute traffic across multiple regions within a geographic area.
3. Use prompt routing to distribute traffic across multiple FMs from the same family.
4. Use provisioned throughput to provision a higher level of throughput for the FM.

20.A travel planning assistant allows users to ask multi part questions such as "Find me family friendly beach destinations near a warm city in April, then suggest three hotels with kids' clubs and create a day by day itinerary." The current implementation sends the raw user query directly into the retrieval layer, which leads to noisy results and incomplete constraint matching. Product management wants the system to better interpret user intent, break complex questions into smaller retrieval steps, and reformulate queries so that more relevant documents are retrieved before generating a final response. How should you structure the query handling and transformation pipeline in this scenario?

1. Call the foundation model first to generate a preliminary itinerary, extract keywords from that output with Lambda, and run a second retrieval step using those extracted keywords. Then use the reranked results to refine the FM response
2. Implement a multi-stage query processing pipeline with AWS Lambda and Step Functions that breaks the user query into intent specific subqueries, performs query expansion using an Amazon Bedrock model, and issues targeted searches through the vector store. Then aggregate the retrieved context and pass the structured results to the foundation model for final synthesis
3. Store every query and response pair from past users in DynamoDB and use a similarity search over past queries to retrieve the closest match. Then use the matched result as a template for constructing a new answer without performing layered query processing.
4. Use a Lambda function to rewrite the user query with an Amazon Bedrock model and then pass the rewritten query directly to OpenSearch Serverless for a single vector similarity call. Then route the top K results to the foundation model without performing any additional decomposition

21.A document analysis system processes very large technical reports and feeds selected content into a foundation model to generate summaries and answers. As document size increases, engineers observe that some responses are incomplete or miss critical sections, especially when long inputs are passed through multi step workflows. Logs show that parts of the input are not consistently reaching the model during peak usage. The team wants to redesign how documents are split and passed to the model so that important context is preserved and truncation related loss is minimized. The solution must scale for heavy workloads and work reliably across diverse document structures. As a GenAI developer, which approach would you recommend to address the content handling issue?

1. Use a semantic chunking strategy that splits documents based on meaning and topic boundaries before generating embeddings with Amazon Bedrock. Pass semantically coherent chunks to the model so each request fits within the context window. Preserve relevance by selecting chunks dynamically at query time
2. Use fixed size token based chunking and rely on prompt compression techniques to reduce overall input length. Truncate lower priority chunks first when token limits are reached. Monitor truncation events using logging services to tune chunk sizes over time
3. Use a semantic chunking strategy combined with static token limits per section and precomputed summaries for every chunk. Concatenate all summaries into a single prompt before model invocation and rely on truncation to discard excess tokens when limits are exceeded. Adjust chunk size thresholds periodically based on average document length
4. Use a hierarchical chunking strategy that divides documents into fixed sections such as chapters, headings, and paragraphs. Aggregate child chunks into parent summaries and pass

the top level summaries to the model. Increase chunk depth to reduce the chance of truncation during inference

22.A customer facing application provides real time AI generated responses as users type queries into a chat style interface. User research shows that perceived responsiveness matters more than receiving the entire response at once, especially for longer answers. The current implementation waits for the full model output before returning a response, which leads to noticeable delays and a poor user experience during peak usage. The engineering team wants to redesign the interaction pattern to deliver the first tokens as quickly as possible while still supporting scalable, managed APIs. They must choose an approach that minimizes end user latency without over engineering the frontend integration. What is the MOST operationally efficient way to improve responsiveness for time sensitive interactions?

1. Use Amazon Bedrock InvokeModelWithStreaming and push tokens to clients using Amazon API Gateway WebSocket API connections. Maintain persistent bidirectional connections and manage connection state for each client. Close the connection after the full response is generated
2. Use Amazon Bedrock InvokeModelWithStreaming to stream tokens as they are generated by the foundation model. Forward the streamed output through Amazon API Gateway HTTP API so clients receive partial responses over HTTP immediately. Optimize performance by reducing payload size and using faster HTTP integrations
3. Use Amazon Bedrock InvokeModelWithStreaming to stream tokens as they are generated by the foundation model. Forward the streamed output through Amazon API Gateway REST API response streaming so clients receive partial responses over HTTP immediately. Avoid buffering responses in the backend to minimize time to first token
4. Use Amazon Bedrock InvokeModel to generate the full response and return it through Amazon API Gateway HTTP API. Optimize performance by reducing payload size and using faster HTTP integrations. Leverage client side loading indicators to manage perceived latency

23.A generative AI platform exposes a public endpoint that accepts free-form user prompts and forwards them to a foundation model. During load testing, engineers observe that some requests fail unpredictably when users submit very long prompts or request verbose outputs. These failures cause retries at the client layer and increase overall latency. The development team wants to introduce controls so that requests exceeding safe limits are handled consistently before reaching the model. The solution must protect downstream services, provide predictable behavior for clients, and avoid embedding complex logic into application code. What do you recommend as the MOST operationally efficient solution for this workload?

1. Use Amazon API Gateway request validation and mapping templates to enforce maximum input size and estimated token limits before invoking the backend. Reject requests that exceed configured thresholds with clear error responses. Forward only validated requests to the model invocation layer
2. Use Amazon API Gateway to forward all requests to the backend and rely on the foundation model to return token limit errors. Capture those errors in AWS Lambda and retry the request with reduced parameters. Return the retried response to the client
3. Use AWS Lambda to calculate token usage for every request after it passes through the API layer. Truncate inputs and outputs dynamically based on remaining context window size. Allow the API to accept all requests without validation to maximize flexibility
4. Use Amazon API Gateway to enforce payload size limits and additionally implement custom token counting logic in Lambda to dynamically rewrite prompts and outputs, truncate content when limits are exceeded, and retry requests automatically until they succeed, while logging all adjustments for audit purposes.

24.A company operates a generative AI system that produces written recommendations used by different internal teams. During review cycles, stakeholders notice that certain phrasing patterns and contextual cues in the inputs and outputs can unintentionally influence how recommendations are framed. The team wants to introduce systematic controls so that inputs are normalized before inference and outputs are consistently adjusted before being consumed by downstream systems. The team also wants to automate comparisons without embedding complex logic directly into application code. The solution must support controlled experimentation, allow teams to compare different prompt handling strategies, and ensure that any pre processing and post processing steps are applied uniformly across evaluations. What do you recommend?

1. Use Amazon Bedrock Prompt Management to maintain multiple prompt versions, attach metadata for fairness experiments, and leverage downstream systems to interpret and adjust outputs. Coordinate evaluations by switching prompts and correlating results manually across test runs
2. Use Amazon Bedrock Prompt Flows to define a multi step workflow that applies standardized pre-processing to inputs and post processing to outputs. Run A/B evaluations by comparing

different flow variants and capture metrics automatically. Use the flow structure to ensure consistent handling across all evaluations
3. Use Amazon Bedrock Prompt Management to version prompts and manually apply input normalization and output rewriting inside the prompt text. Switch prompt versions during testing to compare results across groups. Log outputs externally for later fairness analysis
4. Use Amazon Bedrock Prompt Flows to orchestrate model calls, but embed all pre processing and post processing logic inside application code before invoking the flow. Use the flow to route requests and capture outputs. Compare results by toggling code paths during tests

25.A generative AI analytics application queries a centralized data lake that contains sensitive business data shared across multiple teams. Different users are allowed to see different values within the same dataset depending on attributes such as geography, role, or business unit. During reviews, the security team finds that coarse access controls expose more data than intended for some users. The platform team wants to enforce fine grained data access so that only authorized values within specific rows and specific columns are visible to each request. The solution must integrate cleanly with existing analytics engines and avoid duplicating datasets or building custom filtering logic in application code. What do you recommend?

1. Use AWS Lake Formation to define cell-level security policies that restrict access to specific combinations of rows and columns based on user attributes. Grant permissions through Lake Formation so query engines enforce these rules automatically at query time
2. Use AWS Lake Formation to apply column-level permissions and combine them with row-level locking semantics to prevent unauthorized access. Configure analytics engines to respect these locks during query execution. Manage exceptions through IAM policies
3. Store data in Amazon S3 tables and apply IAM policies that restrict access to specific object prefixes representing rows and columns. Enforce access control by reorganizing data layout and controlling S3 permissions. Update application queries to target only allowed prefixes
4. Use AWS Lake Formation to grant column-level permissions and require application queries to apply WHERE clauses that filter rows dynamically based on user identity. Validate access by reviewing query logs and audit trails. Enforce consistency through development guidelines

26.A healthcare provider manages a large library of patient education material that is frequently revised based on new clinical guidance, regulatory updates, and quality reviews. Some updates involve minor text edits that should trigger lightweight incremental updates to the vector store, while more urgent corrections must update embeddings immediately to avoid distributing outdated information to patients. Additionally, the provider requires a complete vector index rebuild every quarter to eliminate drift, fragmentation, and outdated embeddings. What maintenance architecture should the team deploy to keep the vector store current? (Select two)

1. Use Amazon EventBridge rules and AWS Lambda to process incremental updates and real-time corrections
2. Use an hourly AWS Batch job to reprocess all documents and regenerate embeddings, with manual triggers for urgent updates
3. Use Amazon Bedrock Knowledge Bases with a quarterly document sync job for full index rebuild
4. Use an EventBridge scheduled rule and an AWS Step Functions workflow to perform quarterly full index rebuilds
5. Use Amazon OpenSearch Service ingestion pipelines for all changes and rely on scheduled cron based reprocessing for full index replacement

27.A financial advisory firm is piloting an Amazon Bedrock agent that recommends investment strategies based on a retrieval augmented knowledge base and several downstream APIs. Their internal audit team wants a detailed view of every reasoning step the agent takes, including retrieval calls, tool invocations, decision points, and intermediate conclusions, so that they can trace how any recommendation was derived during post incident reviews. How should you design the tracing and logging architecture so that auditors can reconstruct the full agent reasoning path for any given interaction?

1. Enable agent tracing in the Amazon Bedrock InvokeAgent API and capture all reasoning steps, retrieval metadata, and tool invocation details in structured CloudWatch Logs. Store trace events in a consistent schema and integrate them with CloudWatch Logs Insights so auditors can replay the full reasoning sequence for any historical recommendation
2. Configure Amazon Bedrock Guardrails to enforce reasoning visibility and send filtered reasoning summaries to a Lambda function for logging. Use CloudWatch metrics to track when reasoning content exceeds guardrail thresholds and rely on these summaries for audit reconstruction
3. Enable agent tracing in the Amazon Bedrock InvokeAgent API and record the summarized reasoning output into CloudWatch Logs for downstream review. Use CloudWatch dashboards to visualize high level agent activity to understand the full reasoning sequence

4. Use a Lambda based ingestion pipeline to capture model inputs and outputs and store them in CloudWatch Logs without enabling agent tracing. Add CloudWatch alarms to notify auditors when model outputs require review and depend on application logs to infer how the agent reached its conclusions

28.A large enterprise is preparing to deploy an advanced language model that is several hundred gigabytes in size so that all inference happens entirely within its private network boundary. Initial tests show that the model files and tokenizer components take a long time to download and load into GPU memory, causing container startup to fail and preventing the system from becoming ready. The operations team also needs reliable and predictable health checks so that deployments do not roll back during long initialization periods. How would you configure the hosting environment, container behavior, and initialization timeouts to ensure that this very large model can load successfully and serve real time traffic?

1. Use SageMaker AI real time inference with medium sized GPU instances and shorten container startup timeouts so that failed containers restart quickly. Rely on repeated restarts to eventually complete the model load when the infrastructure is under low utilization
2. Use SageMaker AI real time inference with large GPU instances such as ml.p4d or ml.g5 and configure extended container startup and health check timeouts to allow long model loading periods. Increase model loading quotas and adjust container health checks so that the endpoint becomes ready only after the model is fully downloaded and initialized
3. Deploy the model on SageMaker AI multi model endpoints that automatically load models on demand to reduce initialization time. Let the container evict and reload model files dynamically to optimize for throughput and faster startup
4. Use SageMaker AI real time inference with large GPU instances such as ml.g5 and keep default container startup and health check timeouts unchanged. Rely on automatic retries during deployment to allow the model to finish loading without adjusting the initialization window

29.A public sector agency uses a foundation model to draft citizen communication templates in multiple languages. Policy stakeholders are concerned that tone, content, and examples might differ unfairly across demographic groups, and they want measurable fairness metrics with automatic alerts when bias exceeds agreed thresholds. As the GenAI reliability engineer, how should you implement a solution that continuously evaluates outputs across protected groups and flags regressions?

1. Implement a Step Functions workflow that periodically samples outputs and stores them in CloudWatch Logs for human review. Add a CloudWatch alarm based on log volume changes and depend on manual inspection to identify potential fairness drift
2. Create CloudWatch metrics that store aggregated demographic sentiment and tone scores for each output and rely on a single Lambda function to push these values. Use CloudWatch alarms to notify administrators when overall sentiment trends shift
3. Define custom fairness metrics in Amazon CloudWatch that track bias indicators across demographic groups and integrate a fairness evaluation Lambda function that computes the metrics for each model output. Configure CloudWatch alarms and dashboards to alert reviewers when fairness thresholds are exceeded and trigger automated workflows for regression analysis
4. Use Amazon Bedrock Guardrails to block outputs that contain sensitive or demographic specific language and log guardrail violations into CloudWatch metrics. Configure alarms that notify reviewers whenever the number of blocked messages reaches a threshold and use this as an indicator of fairness performance

30.A digital assistant answers user questions across multiple business functions, including customer support, product usage, and general knowledge. During internal reviews, teams observe that some user inputs contain aggressive or explicit phrasing that varies in severity, while other inputs request guidance in domains, such as investment advice, that the company has decided the assistant should never address, even when the request is phrased in a neutral or professional manner. The platform team wants to enforce consistent safeguards at inference time. The solution must handle both types of inputs predictably, support centralized policy updates, and apply controls automatically during model invocation. Which design choices should be combined to provide the MOST appropriate solution? (Select two)

1. Use Amazon Bedrock Guardrails content filters to evaluate prompts and responses for unsafe language patterns such as harassment, sexual references, or violent expressions. Configure category specific thresholds to determine when responses should be filtered, masked, or refused. Apply these checks during inference to enforce consistent behavior
2. Use Amazon Bedrock Guardrails content filters for language analysis and add a custom Lambda based topic classification layer that evaluates intent using predefined domain taxonomies, synchronizes rule updates across environments, and coordinates enforcement decisions through Step Functions prior to invoking the model
3. Use Amazon Bedrock Guardrails with the strictest content filter settings across all categories and treat requests related to restricted domains as high severity language violations. Tune

thresholds over time based on observed user behavior and false positives. Leverage filter severity to suppress all undesired responses

4. Use Amazon Bedrock Guardrails denied topics to define subject areas the assistant must not address under any circumstances. Detect requests related to those domains regardless of tone or wording. Return deterministic refusal responses when such requests are identified
5. Use AWS Step Functions and AWS Lambda to orchestrate a multi stage validation pipeline that classifies prompts by sentiment and topic intent, applies business logic rules, invokes guardrails conditionally, and routes decisions through approval states before model execution

31.An educational company has a multi-step GenAI pipeline that generates quiz questions from course materials, validates them, and publishes them to a learning platform. The pipeline sometimes fails due to malformed source content, model timeouts, or quota issues. The company wants end to end visibility into where failures occur, automatic retries with backoff for transient errors, routing of permanently failed items to a dead- letter queue, and centralized logs and metrics for troubleshooting. How would you orchestrate the GenAI workflow to provide resilience, observability, and targeted error handling across all steps?

1. Implement an AWS Step Functions Standard workflow that wraps each GenAI step in states with Retry for transient errors and Catch that routes unrecoverable failures to an SQS dead letter queue. Enable CloudWatch Logs and Metrics for the state machine and integrate AWS X Ray tracing on Lambda tasks and model calls to gain full visibility into latency and failure patterns
2. Create an Amazon SQS queue that feeds a fleet of EC2 worker instances running a custom GenAI processing script. Push application logs to CloudWatch Logs for troubleshooting
3. Use Amazon EventBridge to trigger a series of independent Lambda functions chained by events. Configure each Lambda function to write status messages into an S3 bucket. Leverage ad hoc log searches in CloudWatch Logs and manual inspection of the S3 status files to identify and retry failed items
4. Place all GenAI logic into a single Lambda function that processes each item end to end. Leverage Lambda automatic retries and a dead letter queue for failed invocations. Configure basic CloudWatch alarms on function errors for monitoring and observability

32.An e commerce company uses a foundation model on Amazon Bedrock to generate product recommendations and is evaluating a new candidate model that might improve conversion rates. The company wants to run an online A/B test that sends a small percentage of traffic to the candidate model, tracks detailed metrics for each model, and optionally combines outputs with an ensemble strategy before responding to the user. How should you design the A/B testing and ensemble routing pattern for these foundation models?

1. Use Amazon Route 53 weighted routing to send 10 percent of traffic to a separate API endpoint that fronts the candidate Bedrock model and 90 percent to the existing endpoint, without any additional routing logic in the application. Rely on CloudWatch aggregated metrics at the endpoint level and manually compare overall performance to decide when to switch all traffic to the candidate model
2. Use Amazon API Gateway and AWS Lambda to randomly route about 5 to 10 percent of requests to the candidate Bedrock model while sending the rest to the existing model, driven by a traffic split configuration stored in AWS AppConfig. Tag each request with the selected model identifier and record conversion, latency, and quality metrics per model in Amazon CloudWatch and Amazon DynamoDB, optionally running ensemble logic in the Lambda function that can be enabled or disabled without redeploying
3. Use Amazon API Gateway with a Lambda authorizer that reads a feature flag from AWS AppConfig and calls both Bedrock models for every request, then returns the candidate model output to the client while storing the primary model output only for offline analysis. Aggregate all performance metrics into a single CloudWatch metric namespace without including the model identifier to simplify reporting
4. Update the recommendation service to route all online traffic to the candidate Bedrock model during the test period while periodically invoking the original model in an offline batch job for comparison. Store experiment results in Amazon S3 and use scheduled AWS Glue jobs to compute conversion statistics before deciding whether to roll back or keep the candidate model

33.A global knowledge-management platform allows users to perform diverse tasks such as contract summarization, marketing copy generation, policy rewriting, and troubleshooting support queries. Since each task demands different quality-latency- cost tradeoffs, the engineering team wants to route incoming prompts to the most suitable foundation model without requiring users to choose a model manually. The team is evaluating routing mechanisms that can intelligently inspect the prompt, classify the task category, and forward the request to the optimal LLM while maintaining a single unified interface for all use cases. Which routing strategy should the team implement?

1. Implement a rules based classifier that evaluates prompt keywords and routes requests to predefined LLMs without using a model driven classification step
2. Use a small generative model to rewrite incoming prompts into standardized task formats and forward all rewritten prompts to a single downstream LLM
3. Use a single general purpose LLM for all tasks and leverage prompt templates to adapt behavior
4. Implement an LLM-assisted router that uses a lightweight classifier model to examine the prompt, identify the task type, and route it to the most appropriate downstream LLM

34.A healthcare information portal is rolling out a GenAI assistant for clinicians that summarizes guidelines and suggests differential diagnoses. Clinicians report that the assistant provides final answers without revealing the reasoning behind them, and they want visibility into intermediate steps, confidence indicators, and the sequence of decisions taken as the response is generated. As the GenAI experience owner, how would you redesign the interaction pattern and user interface to make the assistant's decision process more transparent?

1. Use Amazon Bedrock streaming to deliver intermediate reasoning steps as they are generated and render these steps in the UI through a structured reasoning display component. Configure structured output prompts in Prompt Flows that extract reasoning chains and confidence indicators so clinicians can see how each conclusion is formed
2. Create a Lambda function that post processes model outputs to add generic disclaimers and a static confidence score before sending results to the UI. Store these generated summaries in CloudWatch Logs for later transparency reviews
3. Use Amazon Bedrock streaming to deliver the model's full response in smaller chunks and present it incrementally in the UI. Leverage CloudWatch metrics to track confidence levels in the background instead of exposing step by step reasoning directly to clinicians
4. Configure Amazon Bedrock Guardrails to block incomplete or low confidence outputs and show only fully validated responses to clinicians. Use CloudWatch alarms to notify administrators whenever reasoning steps are filtered during generation

35.A media company’s content generation system must remain available across multiple geographic regions. During high traffic periods, Amazon Bedrock occasionally returns “Too many requests” errors for their preferred model. The company wants to increase resilience without modifying the application’s content generation logic or switching to a different foundation model family. They want an approach that preserves model consistency while ensuring that surges in regional traffic do not cause user facing failures. Which model configuration strategy should they choose?

1. Enable Amazon Bedrock cross Region inference so that requests automatically fail over to another Region when the primary Region experiences throttling
2. Use Amazon CloudFront to route inference traffic through an edge distribution layer before the requests reach Amazon Bedrock from all Regions
3. Increase the maximum number of provisioned inference units for the model in a single Region and rely on autoscaling to handle peak load coming from other Regions
4. Configure a Step Functions workflow that retries failed model invocations with exponential backoff and replays requests until the throttling condition clears

36.A financial services company is building an internal GenAI assistant on Amazon Bedrock that will process highly sensitive transaction narratives and dispute descriptions. The security team requires that all Bedrock traffic stays entirely within their AWS network and that only specific application subnets can call the Bedrock APIs, with no exposure to the public internet. How should you design the network architecture to meet these isolation requirements while still allowing the assistant to scale across multiple Availability Zones?

1. Use a NAT gateway for all outbound Bedrock API calls and restrict NAT access to only the application subnets. Enable cross Availability Zone failover by attaching additional route table entries that direct Bedrock traffic through the NAT infrastructure
2. Create a VPC endpoint service for the application and attach security groups that restrict which subnets can connect while routing Bedrock traffic through the service. Configure the assistant to access Bedrock using internal DNS rules so that traffic does not require direct public connectivity
3. Configure interface VPC endpoints for Amazon Bedrock using AWS PrivateLink and attach tightly scoped security groups that allow only approved application subnets to connect. Deploy the GenAI assistant across multiple Availability Zones to ensure scalable and private connectivity without any public internet access
4. Create interface VPC endpoints for Amazon Bedrock and attach security groups that allow access from all subnets in the VPC for simplified management. Deploy the assistant across multiple Availability Zones and rely on default VPC routing to maintain private connectivity

37.An analytics organization has accumulated years of customer feedback distributed across hundreds of storage locations, with no clear inventory of which files contain sensitive details such as

personal identifiers, financial information, or regulated content. The GenAI development team wants to build both fine tuned models and retrieval pipelines from this data, but the security team insists on gaining automated visibility into the presence of sensitive information before any dataset is used for GenAI processing. Leadership also wants a comprehensive method to categorize, protect, and track risky files so that only approved datasets flow into AI workflows. What do you recommend for the given use case?

1. Use Amazon Athena to query sample objects from S3 and rely on pattern matching queries to estimate which files might contain regulated fields. Push all candidate datasets to a staging bucket and use IAM conditions to control downstream access during model training
2. Enable Amazon Macie with default monitoring. Leverage the automated object sampling to identify sensitive files in the S3 environment efficiently. Use the partial findings to label only the detected objects and allow the remaining unscanned data to flow into fine tuning and retrieval pipelines.
3. Enable Amazon Macie across the relevant S3 buckets and configure automated sensitive data discovery jobs for full discovery to scan objects for PII and regulated patterns. Use the classification findings to tag, quarantine, or restrict access to high risk objects before allowing them into fine tuning or RAG workflows
4. Configure Amazon Bedrock Guardrails to block sensitive categories and rely on the guardrail rules to detect and prevent PII from entering fine tuning or retrieval datasets stored in S3. Allow the RAG pipeline to ingest all S3 objects and depend on the guardrail filtering layer to ensure that only compliant data is surfaced to the model

38.A global automotive manufacturer is developing a unified "Product Intelligence Assistant" that can help engineering, marketing, and field-service teams search across product manuals, assembly-line videos, test-drive transcripts, and CAD-annotation metadata. The assistant must support multimodal retrieval, provide streaming conversational responses with grounded citations, and route requests to different foundation models depending on the requesting business unit. Additionally, the company must strictly separate pre-release and public content and wants to avoid the operational burden of building custom multimodal ingestion pipelines or hosting models directly. What do you recommend?

1. Use Amazon Bedrock Knowledge Bases to ingest multimodal text, transcripts, and image-derived embeddings into separate knowledge bases for public and pre-release content. Then use Amazon Bedrock Flows to orchestrate model- specific routing based on the requesting business unit, combine the retrieval output with Titan or Claude models, and stream responses to applications through the Amazon Bedrock Converse API
2. Use a single Anthropic Claude model in Amazon Bedrock and configure a Lambda function to classify business-unit requests. Pass all multimodal content as pre- processed text to the model and store semantic metadata inside DynamoDB
3. Use an Application Load Balancer with ECS services that host model adapters. Implement a custom retrieval engine using OpenSearch and a global router using API Gateway and Lambda. Enable per-unit isolation with IAM condition keys
4. Deploy custom multimodal models on Amazon SageMaker, create a Step Functions workflow to handle text/image/video processing, and implement conditional branches for each business unit

39.A SaaS support assistant is producing outdated recommendations even though the documentation repository has already incorporated the latest troubleshooting procedures and removed obsolete content. A review of the pipeline shows that only a portion of the corpus has been re-embedded using the new embedding model and preprocessing logic, while many older vectors remain unchanged. How should you identify and correct embedding drift so that search results consistently reflect the most up to date knowledge?

1. Rebuild the entire vector store using Amazon Bedrock Knowledge Bases with a single Titan Embeddings model and uniform preprocessing rules to eliminate version mismatches. Run retrieval relevance checks and drift evaluation tests to confirm that all vectors are aligned with the updated corpus
2. Re-embed only the documents that changed in the recent update cycle and preserve all older vectors generated with the previous embedding model. Modify OpenSearch ANN search settings to reduce stale recommendations without altering the embedding workflow
3. Use Amazon Bedrock Knowledge Bases but retain the mixed set of legacy and new embeddings while applying metadata filters that boost recently updated articles. Depend on Bedrock model based re-ranking to adjust for inconsistencies between the different embedding generations
4. Perform a partial refresh by re-embedding only the segments related to newly added troubleshooting steps while leaving legacy embeddings in place. Validate retrieval quality using targeted tests rather than performing a full corpus re- embedding

40.A media platform uses generative models to produce large volumes of content recommendations and short summaries. Manual review for fairness and stereotyping has become impractical, but the trust and safety team still needs a consistent evaluation approach that identifies biased or unfair patterns across demographic groups with periodic human spot checks. As the responsible AI specialist, how would you design an automated judging system that evaluates fairness, reduces judge model bias, and remains aligned with human reviewers?

1. Design an LLM as a judge workflow by defining structured fairness rubrics and evaluating each generated output through Amazon Bedrock Prompt Flows that orchestrate fairness scoring. Use ensemble judging across multiple Bedrock models and regularly calibrate judge scores against a human labeled dataset to reduce bias and maintain reviewer alignment
2. Create an LLM as a judge workflow by defining fairness scoring rubrics and evaluating each output through Amazon Bedrock Prompt Flows that run a single high performing foundation model. Periodically compare judge results with historical fairness thresholds and adjust scoring prompts instead of performing calibration with human labeled data
3. Use Amazon Bedrock Guardrails to block outputs that contain demographic references or potentially sensitive phrases. Enable CloudWatch alarms so reviewers can examine filtered responses and treat filter triggers as evidence of fairness compliance
4. Implement a fairness judging pipeline by defining scoring prompts and routing generated outputs through a single model in Amazon Bedrock Prompt Flows. Leverage CloudWatch metrics to validate judge consistency over time instead of performing calibration with human labeled samples

41.A large manufacturing company is experiencing a significant increase in unplanned downtime due to recurring production incidents. Each incident typically requires analyzing large volumes of sensor data, reviewing historical maintenance logs, and determining the most effective remediation plan. The engineering leadership wants to automate this multi step diagnostic workflow using a set of specialized AI agents that can work together, exchange findings, and dynamically delegate parts of the investigation based on their expertise. As a GenAI developer, which approach should you implement to enable specialized agents to collaborate and coordinate effectively during root cause analysis? (Select two)

1. Configure AWS Strands Agents for each specialization and use custom API Gateway endpoints for agents to exchange findings for centralized coordination system
2. Configure three AWS Strands Agents with their specialized capabilities but run them as independent agents without an orchestration layer. Instruct the application to call each agent sequentially and combine their outputs manually without enabling direct collaboration or shared state between them
3. Use AWS Agent Squad to orchestrate collaboration so that agents can exchange intermediate findings and dynamically delegate subtasks during incident analysis
4. Configure specialized AWS Strands Agents for sensor analysis, maintenance log interpretation, and remediation planning, and attach clear role definitions and tools to each agent
5. Configure a single Amazon Bedrock Agent with multiple action groups that perform sensor inspection, log parsing, and remediation recommendations within one agent. Add prompt engineering rules that instruct the agent to self coordinate expertise across these domains without using multi agent collaboration features

42.A legal tech platform wants to measure accuracy, robustness and reasoning quality across multiple model candidates. The team must approve new foundation models into production only after they pass a consistent and repeatable evaluation cycle. They need an automated mechanism that applies controlled metrics, compares multiple candidates side by side, and produces quantifiable scores for decision making. Which evaluation approach is most suitable for these requirements?

1. Use Amazon Bedrock Model Evaluation to automatically score multiple model candidates against a curated evaluation dataset
2. Use an internal human review process where domain experts score outputs from each model candidate. Store reviewer feedback in a structured format and compute aggregate ratings before approving models for production
3. Use a Lambda powered A/B testing framework that routes traffic between multiple model candidates and collects user feedback at runtime. Use this feedback to compute the metrics and choose the highest rated model for production approval
4. Use CloudWatch Logs and embed evaluation metadata inside inference logs so that engineers can manually compare hallucination incidents and summarization differences across models. Generate nightly reports from the aggregated logs to decide which model to promote

43.An HR SaaS provider is developing multiple GenAI capabilities on Amazon Bedrock, including job description generation, interview question drafting, and performance review summarization. Product teams have begun creating and modifying system prompts independently, resulting in inconsistent tone, duplicated effort, and uncontrolled changes moving into production. Leadership wants to establish a centralized prompt catalog with parameterized templates, role based approval workflows,

and full auditability of prompt changes across teams. As a GenAI developer, how should you implement prompt management and governance so that teams can safely reuse and evolve prompts?

1. Use Amazon Bedrock Prompt Management to create parameterized templates, configure role based approval workflows, and enable version tracking for all prompt updates. Store final approved templates in S3 for application level consumption
2. Use Amazon Bedrock Prompt Management to store prompt templates and allow teams to update versions automatically when changes are detected. Use CloudWatch to record which prompt versions are used in production
3. Use DynamoDB to maintain prompt templates and use Step Functions to approve or reject updates. Track template history with CloudWatch Logs for auditability
4. Store all prompt templates in S3 and use AWS IAM permissions to control which teams can modify them. Enable CloudTrail to track file access and rely on Lambda functions to validate template structure

44.A marketing content assistant recently adopted a new “tone and brand” prompt template that was manually tuned in a staging environment. After rollout, some product lines show improved copy while others regress, and the team has no structured way to compare the new template against the previous one across a broad test set. How should you introduce a prompt testing framework and evaluation workflow to systematically compare prompt versions and prevent quality regressions?

1. Use Amazon Bedrock model evaluation jobs to run both the legacy and updated prompt templates across a comprehensive validation dataset and capture structured quality metrics. Store prompt versions and evaluation outputs in Amazon DynamoDB so that you can automate periodic comparisons and detect regressions consistently
2. Use Amazon Bedrock to execute both prompt versions on a small set of sample inputs and manually compare the outputs for tone and brand alignment. Save the sample outputs to Amazon S3 and perform occasional reviews when product teams raise concerns
3. Use Amazon Bedrock model evaluation jobs to test only the updated prompt template against a curated dataset while retaining the previous template as a baseline stored in DynamoDB. Review the evaluation outputs periodically and adjust the new template based on high level scoring trends rather than running side by side comparisons
4. Adopt the new prompt template only for recently introduced products and phase it into older product lines over time. Use S3 object versioning to track prompt templates and assume the new configuration maintains quality across product segments without structured testing

45.A supply chain organization has built a graph of suppliers, warehouses, routes, and risk scores and is now adding textual reports, incident summaries, and audit findings to help a foundation model answer risk related questions. The team wants the assistant to consider both the graph structure and the semantic similarity of text when retrieving context for questions like "Which suppliers are at highest risk of disruption over the next quarter, and why?" As the GenAI specialist, you must design a retrieval mechanism that can perform vector based top K searches over graph attached text while still respecting relationships in the graph. What is the most effective approach to design this graph aware retrieval strategy for FM augmentation?

1. Store the graph in a separate database and export all node and edge data into a vector store so the system can run a global vector search across all content. Then attach graph metadata to the top K vector results to approximate graph awareness during post processing
2. Run vector similarity searches in a stand alone vector database and then filter the top results by querying the graph database separately. Then merge both results client side before sending them to the foundation model to provide contextual
3. Integrate the graph database with a vector capable analytics engine that supports top K similarity search on embedded text stored as node or edge properties. Then run vector similarity queries inside the graph engine so the retrieval respects graph relationships before passing results to the foundation model
4. Convert the entire graph to flattened documents and generate embeddings for each combined record so the assistant can perform a single vector similarity search. Then rely on the foundation model to infer supply chain relationships directly from the flattened text

46.A customer support organization receives thousands of unstructured issue descriptions each day, many of which contain sensitive personal details such as phone numbers, home addresses, and customer account identifiers. The team wants to build an automated summarization workflow that can generate concise and accurate summaries without exposing the raw sensitive data to the language model. The leadership mandates that sensitive elements must be identified, transformed, or removed before the summarization step, and that downstream teams should be able to audit how personal data was handled. Which of the following represents the best solution?

1. Integrate Amazon Comprehend PII detection into the workflow and tag detected entities. Keep the original text intact so the foundation model can use full context during summarization.

Store the redaction metadata separately and rely on downstream systems to mask sensitive fields after the model completes its output
2. Integrate Amazon Comprehend PII detection as a preprocessing step and use a Lambda function to automatically redact or replace detected PII entities before passing the sanitized text to the foundation model. Configure the workflow so that only redacted versions are forwarded while logging detection metadata for compliance audits
3. Implement a data masking process using CloudWatch Logs Insights that scans stored support tickets and replaces sensitive values after the content has already been processed by the model. Forward the model generated summaries to downstream systems without additional filtering steps
4. Use Amazon Comprehend to detect PII entities and send both the redacted and original text to the foundation model so that it can choose the best version for summarization. Store the original content in a secondary location and rely on IAM policies to prevent unauthorized access

47.A media company wants to add a conversational assistant into its existing customer portal serving both web and mobile clients. The solution must return real-time responses, support GraphQL queries that fetch and update domain data alongside foundation model outputs, and convert incoming GraphQL requests into structured prompts for the foundation model while keeping latency low and hiding model API details from client applications. Which solution would you implement to address these requirements?

1. Use AWS AppSync with pipeline resolvers that invoke a Velocity Template Language (VTL) template to forward GraphQL requests directly to the foundation model endpoint. Rely on AppSync caching to reduce latency and expose the model output to clients without using Lambda for prompt transformation
2. Use AWS Step Functions to orchestrate GraphQL parsing and foundation model calls and return results through Amazon EventBridge. Integrate the customer portal with EventBridge to fetch conversational responses from subscribed event targets
3. Use AWS AppSync with direct Lambda resolvers to process GraphQL queries and transform them into foundation model prompts. Implement the inference logic inside the Lambda function and return structured outputs to clients while keeping model details hidden
4. Use an API Gateway HTTP API fronting a Lambda function that parses GraphQL payloads and invokes the foundation model endpoint. Cache responses in API Gateway to reduce model invocations and expose the HTTP API directly to web and mobile clients

48.A media intelligence company processes millions of global news articles each day and provides real-time insights to analysts, advertisers, and policy research teams. To support fast and accurate information retrieval, the platform needs a vector search solution that can segment content by topic, combine semantic relevance with keyword precision, and accelerate lookup using approximate nearest neighbor search. The engineering team must also tune index mappings, shard counts, and replica strategies to handle query spikes during major events. Which solution should the team choose to meet these requirements?

Use Amazon OpenSearch Service and store all documents in a single global index with default mappings and no topic-specific partitioning. Create one large index for all news articles and rely on OpenSearch defaults for mapping, shard configurations, and keyword fields. Use exact k-NN search across the entire dataset for semantic retrieval

Configure Amazon OpenSearch Service with the Neural Search plugin. Deploy the Neural Search plugin to enable hybrid semantic and keyword retrieval across topic-specific indices. Configure ANN-based acceleration, shard tuning, and optimized mappings to reduce latency under concurrent load

Use Amazon Athena federated queries to search enriched metadata and perform semantic lookups through an external embedding API

Store all documents in a Bedrock Knowledge Base and rely on metadata tagging to differentiate topics and improve retrieval quality. Use the built-in retrieval flow to handle semantic relevance and indexing requirements

49.A global retailer is developing a GenAI powered product assistant that must answer questions about product specifications, warranty rules, compatibility details, and regulatory standards drawn from thousands of internal documents. Early testing shows that the model often fabricates details that do not appear in the authorized documentation, which results in inaccurate answers being shown to customers. The engineering leadership wants every response generated by the assistant to be grounded in verified enterprise documents, with a mechanism that checks whether the final output truly aligns with trusted information before returning it to the user. What do you suggest?

1. Use Amazon Kendra as the primary retrieval layer and allow the LLM to incorporate retrieved passages into a free form response without enforcing alignment. Rely on CloudWatch alerts to detect when the model produces hallucinated details
2. Use Amazon Bedrock Knowledge Bases to index documents but allow the model to respond directly without retrieval based grounding. Apply a post processing Lambda function that logs

fabricated answers for later review

3. Use Amazon Bedrock Knowledge Bases to retrieve authoritative product data and ground model responses, then apply confidence scoring and semantic similarity checks to verify alignment before returning the final answer. Configure the system to return only evidence backed responses when grounding confidence is high
4. Use Amazon Translate followed by a summarization model to generate simplified versions of the product documents and instruct the model to answer only from those summaries. Depend on model guidelines and prompt instructions to reduce hallucinations

50.A multinational bank is designing a compliance-focused document review assistant for its risk division. The tool must generate grounded explanations with paragraph-level citations, enforce blocked topics using safety controls, and ensure that any new prompt template undergoes a formal approval workflow before being deployed. The bank also mandates that all model invocations be retained for ten years due to regulatory laws and that the entire system operate with minimal custom infrastructure. Which of the following represents the best solution for the given use case?

1. Use SageMaker hosting with a custom LLM, implement guardrails inside the inference container, store logs in OpenSearch for 10 years, and manage prompt versioning manually via DynamoDB
2. Use Amazon Bedrock Knowledge Bases with Titan Text models, tag documents in S3 with business-unit identifiers, and rely on CloudTrail Data Events for storing a 10- year audit trail
3. Use Amazon Bedrock Prompt Management to enforce multi-stage approval workflows across business units, combine Amazon Bedrock Guardrails for topic blocking and injection resistance, and integrate Amazon Bedrock Knowledge Bases for retrieval with citation metadata. Then enable Amazon Bedrock model invocation logging with Amazon S3 Object Lock configured at 10-year compliance mode retention
4. Use AWS Step functions to enforce multi-stage approval workflows across business units, combine Amazon Bedrock Guardrails for topic blocking and injection resistance, and integrate Amazon Bedrock Knowledge Bases for retrieval with citation metadata. Then enable Amazon Bedrock model invocation logging with Amazon S3 Object Lock configured at 10-year governance mode retention

51.A retail analytics company operates a recommendation API that relies on Amazon OpenSearch Service to provide real-time product suggestions. The system stores more than 8 million vector embeddings and processes thousands of concurrent queries during peak shopping periods. Recently, the development team has observed significant query latency and inconsistent response times due to index size, suboptimal shard configurations, and insufficient optimization of approximate nearest neighbor (ANN) search parameters. Which optimization strategy should the team adopt? (Select two)

1. Migrate all vector embeddings to Bedrock Knowledge Bases and use managed retrieval to replace the OpenSearch index
2. Replace ANN based search with exact k-NN search on the existing single index and increase cluster storage capacity
3. Segment the dataset into multiple specialized indices based on product category and apply ANN tuning and shard resizing per index
4. Reconfigure the OpenSearch cluster with additional shards, optimized replica counts, and suitable instance types, and tune ANN parameters such as ef_search and ef_construction
5. Reduce the number of shards to a single global shard and disable replicas to consolidate compute resources onto fewer nodes

52.A financial planning startup wants to build a customer facing GenAI assistant that can remember user preferences across multiple chat sessions, such as risk appetite, preferred investment horizon, and previously approved recommendations. Which solution should you implement?

1. Configure an Amazon Bedrock Agent with action groups and use a single DynamoDB table to store both short term and long term memory items in the same partition key. Add Lambda resolvers to retrieve chat history on each new interaction without separating session context from persistent preferences.
2. Configure an Amazon Bedrock Agent with action groups mapped to the portfolio APIs and implement a memory design that stores short term chat context in a session scoped DynamoDB item and long term user preferences in a separate DynamoDB table. Implement retrieval and update logic in the agent configuration so that the agent consistently maintains user state across sessions
3. Configure a Bedrock Agent without action groups and store short term context in AWS Secrets Manager and long term user profile data in AWS Systems Manager Parameter Store. Use Session Manager to retrieve context and pass it back to the agent during each request
4. Use AWS Step Functions to store both short term and long term memory in state variables and invoke a Bedrock model directly from a Lambda function. Update memory values only during workflow transitions instead of persisting them separately for each user

53.A financial services company is rolling out dozens of foundation models for different risk scoring and customer analytics use cases. The head of model risk management has asked you to ensure that every model promotion through the CI/CD pipeline automatically produces an up to date, regulator ready model card that captures intended use, limitations, key metrics, and risk ratings without manual edits. As a GenAI developer, what should you design so that programmatic model card generation is embedded as a mandatory step in the deployment workflow?

1. Integrate the SageMaker Model Card APIs into the CI/CD pipeline so that deployment stages automatically call create_model_card and update_model_card with metadata extracted from training and evaluation jobs. Configure the pipeline to block model promotion until the generated model card artifacts are validated and stored in the model registry
2. Configure the CI/CD pipeline to run SageMaker Processing jobs that export model evaluation metrics and store them in Amazon S3, then manually trigger model card creation from the SageMaker console. Use CloudWatch Events to notify reviewers whenever new metrics are uploaded
3. Create a Lambda function that runs after each deployment to read evaluation metrics from Amazon S3 and send a summary to the compliance team, who then use the SageMaker console to create model cards manually. Configure Amazon SNS notifications so that the Lambda function alerts reviewers whenever a new model version is promoted
4. Add a non blocking stage to the CI/CD pipeline that calls SageMaker Model Card APIs to create or update model cards using default templates without validating captured metadata. Allow the pipeline to continue promoting the model to production even if the model card creation step fails or produces incomplete documentation

54.An AI team is fine tuning a text generation model on proprietary data using Amazon SageMaker. They want an orchestrated pipeline that handles data preparation, training, evaluation, conditional promotion of the model to a registry, and deployment to a managed inference endpoint. The same pipeline should support rolling back to a previous model if metrics regress and should be easily triggered on new training data or configuration changes. Which orchestration approach would you recommend to coordinate this end to end custom GenAI training and deployment workflow?

1. Build an Amazon SageMaker Pipeline that defines the required steps. Configure the pipeline to trigger on the relevant events and roll back to an approved model version when evaluation metrics degrade
2. Use AWS Step Functions to orchestrate Lambda functions that submit SageMaker training jobs, run evaluation scripts, and update endpoints while storing model versions in a DynamoDB table. Configure an EventBridge rule to start the state machine whenever new training data arrives in S3
3. Schedule a SageMaker notebook instance to run a Jupyter notebook that reads new data, starts a training job, computes metrics, and directly updates the production endpoint. Store the active model information in a JSON file in S3 and overwrite it whenever a new model trains
4. Leverage AWS CodePipeline to detect changes in a source repository and run CodeBuild jobs that call SageMaker APIs for training and deployment. Implement evaluation, metric comparison, and rollback behavior entirely in custom shell scripts inside the build stages

55.A large ecommerce business processes thousands of refund requests every day, each involving different rules, exceptions, and special conditions. Refund decisions often require evaluating complex policy criteria, checking the latest order and delivery information, and determining whether an issue should be resolved automatically or escalated to a specialist. The customer support team wants to automate this reasoning process using an intelligent system that can think through the policy logic step by step, call internal order systems when needed, and repeat this cycle until it reaches a clear decision. Which approach should you implement to enable structured reasoning and action execution in an iterative loop until a final refund decision is reached?

1. Design an AWS Step Functions workflow that alternates between a reasoning state powered by a foundation model and an action state that invokes downstream APIs, implementing the ReAct pattern. Configure the workflow to repeat the reasoning and action cycle until the model reaches a final approval or escalation decision
2. Design a Step Functions workflow that uses a single foundation model state to perform both reasoning and API execution in one step. Configure the workflow to run only once per request without iterative reasoning or repeated API calls
3. Use AWS Glue workflows to coordinate reasoning steps and API calls and store intermediate reasoning outputs in S3. Configure Glue triggers to execute each stage of the refund evaluation process sequentially
4. Use a single Amazon Bedrock Agent with multiple action groups that handle both reasoning and API calls within one step. Configure the agent to follow policy prompts and rely on its internal reasoning to reach a final decision without an orchestrated reasoning action loop

56.A global legal consultancy is developing an AI driven clause drafting system that must route every generated clause through mandatory human review. The firm wants attorneys to evaluate each draft, provide structured feedback, and either approve or request revisions. The firm also wants to capture feedback metadata so it can analyze common revision categories, reviewer patterns, and long term trends that influence model improvement. To achieve this, the system must coordinate automated drafting steps, route human review tasks, trigger downstream processing, and store structured review data in a repository suitable for long term analysis. What do you suggest?

1. Use AWS Step Functions to draft clauses and immediately send them to attorneys via an SNS topic without involving Amazon Augmented AI (A2I). Store lawyer feedback in a single S3 prefix and periodically run Athena queries to detect revision patterns
2. Use Amazon Bedrock Agents to manage the drafting and review process and configure Amazon Augmented AI (A2I) only to notify attorneys by email when a clause is ready for review. Store each reviewer comment as a separate S3 object for later analysis
3. Use AWS Step Functions to orchestrate clause generation and route cases to Amazon Amazon Augmented AI (A2I) for human review, configure A2I to send structured reviewer feedback to an Amazon EventBridge bus, and write downstream consumers that store the feedback in DynamoDB with secondary indexes for long term analysis. Implement separate event patterns to handle approvals, rejections, and revision requests
4. Use Amazon Amazon Augmented AI (A2I) to manage both AI drafting and human review and configure A2I to store feedback only in its managed output buckets. Rely on manual exports from the A2I console to perform long term trend analysis

57.A compliance team needs to process tens of thousands of PDF reports each night and generate concise GenAI summaries for auditors. The reports are uploaded to Amazon S3 throughout the day, and the summaries must be ready by the next morning. The team wants to avoid long synchronous API calls, handle failures for individual documents, and automatically retry only failed items without reprocessing the whole batch. How should the solutions architect orchestrate this large scale, batch GenAI summarization workflow?

1. Configure S3 event notifications to trigger a Lambda function for every new PDF and have the function call Bedrock synchronously to generate a summary and store it in S3
2. Create a nightly Step Functions Express workflow that uses a single Lambda task to scan S3 for all new PDFs, call Bedrock in large batches, and write all summaries back to S3. The workflow uses a Retry policy on that Lambda task so any error causes the entire batch job to rerun from the beginning.
3. Configure Amazon S3 to send document upload events to SQS and use an EventBridge schedule to start a Step Functions Standard workflow that consumes the queue each night with a Map state over document identifiers. Each Map item invokes Lambda to fetch the PDF from S3, calls Amazon Bedrock for summarization, writes the summary back to S3, and applies per item retry policies so only failed documents run again
4. Build an AWS Glue job that runs each night, crawls the S3 bucket for new PDFs, and calls Bedrock from within a single Spark driver script that loops over all documents. The script writes summaries back to S3 and reruns the entire Glue job if any error appears in the nightly logs

58.A global ecommerce company is deploying a GenAI powered helpdesk assistant on Amazon Bedrock to handle order, refund, and shipping inquiries across multiple regions. The compliance team requires that the assistant always follow a company approved professional tone, avoid legal advice, and return responses using a strict JSON output schema that downstream systems can reliably process. The engineering team wants a centralized way to define system prompts, follow governance norms, apply behavioral constraints, and enforce consistent output formatting across all microservices that invoke the foundation model. How would you design the model instruction framework and service integration to enforce these requirements across all applications? (Select two)

1. Store all prompt templates in S3 and use IAM policies to restrict which microservices can modify them. Use CloudWatch Logs to review prompt usage and ensure that applications follow the JSON response format
2. Use Amazon Bedrock Prompt Management to host all prompt templates but allow microservices to automatically promote template versions as part of their CI deployments
3. Apply Amazon Bedrock Guardrails to enforce tone requirements and block legal advisory content across all model interactions
4. Use Amazon Bedrock Prompt Management to define parameterized system prompts, apply JSON Schema based output templates, and configure approval workflows
5. Use DynamoDB to store JSON schemas and rely on Lambda functions to inject tone guidelines and filter prohibited content before returning responses. Use Step Functions to orchestrate prompt selection logic for each workflow

59.A global healthcare analytics company operates a clinical summarization system that currently runs in a single geographic region. New regulatory and availability requirements mandate that the

system must remain fully functional even if one region experiences a prolonged outage, while also ensuring that user requests are routed according to regional compliance rules. You are responsible for designing a multi-Region architecture that supports active active or active passive deployments, performs automated health checks, and redirects traffic safely during failures. How should you architect the multi Region integration and routing strategy to meet these resilience requirements?

1. Deploy the Bedrock based inference workflow in two AWS Regions and expose each through separate regional endpoints registered in Amazon Route 53 using failover or latency based routing. Configure Route 53 health checks to monitor each endpoint and automatically redirect traffic to the healthy Region during outages
2. Run the Bedrock workload in one AWS Region and enable cross Region replication for data, while continuing to direct all inference requests through a single Route 53 simple record. Allow DNS caching to manage failover behavior without using health checks or routing policies
3. Create separate Bedrock endpoints in multiple AWS Regions but configure Route 53 with weighted routing that always favors the primary Region regardless of health status. Allow traffic to shift back only after administrators manually update routing weights following an outage.
4. Deploy the Bedrock workflow in two AWS Regions but front both services with a single Route 53 record that uses simple routing and has no health checks enabled. Leverage application clients to detect endpoint failures and manually shift requests to the secondary Region

60.A GenAI documentation assistant is used by hundreds of internal teams, and cost reports show wide variance in token usage across different workloads. Finance requires accurate chargeback visibility across teams, projects, and environments, and leadership wants strict token budgets for experimental workloads while still permitting higher limits for mission critical applications. As the lead GenAI engineer, how would you design a token estimation and tracking mechanism that enforces per tenant token budgets while allowing prioritized workloads to exceed preset thresholds when necessary?

1. Implement a token accounting layer that records estimated token usage for each request in Amazon DynamoDB and forwards periodic summaries to Amazon CloudWatch Logs for reporting. Use AWS Lambda to soft enforce token budgets by allowing all requests to proceed while tagging overage attempts for later review
2. Build a centralized usage tracker that records token estimates in DynamoDB but allow teams to bypass budget checks by passing a priority flag in their API request. Use a simple Lambda based interceptor to send warnings when teams exceed token budgets instead of enforcing hard limits
3. Create a token metering layer that logs input and output tokens for every request through Amazon API Gateway and stores these metrics in Amazon CloudWatch and Amazon DynamoDB for per tenant reporting. Use AWS Lambda to enforce token budgets by evaluating usage before calling Amazon Bedrock and allow elevated limits for critical workloads through configurable policy rules
4. Aggregate cost data from the AWS Billing and Cost Management console and rely on periodic cost allocation tags to approximate per tenant token consumption. Configure Amazon EventBridge rules to notify teams when overall monthly spending trends exceed predefined thresholds

61.A media moderation system runs hourly batch jobs that send tens of thousands of content snippets through a foundation model to detect policy violations. The team currently issues one inference request per item, which leads to underutilized GPU instances, high costs, and jobs that occasionally miss their completion windows during traffic spikes. How should you redesign the workload using batching strategies and high throughput inference configurations to maximize utilization and throughput?

1. Configure a SageMaker real time endpoint with larger instance types and enable request level parallelism by submitting multiple concurrent mini batches across several threads. Rely on endpoint invocation concurrency to simulate high throughput behavior without adopting full batch transform or asynchronous inference
2. Use Amazon SageMaker Processing jobs to distribute the moderation workload across multiple nodes and have each node call the foundation model endpoint in parallel. Rely on the distributed processing cluster to finish the workload faster without changing the per item invocation pattern
3. Configure a high throughput inference setup using Amazon SageMaker batch transform or SageMaker asynchronous inference and submit large batches that group multiple records per request. Use auto scaling rules on the model endpoint to match peak batch volumes and ensure that GPU instances remain fully utilized
4. Increase the number of GPU instances on the existing real time endpoint and scale them to the maximum size available. Rely on the additional capacity to absorb hourly traffic spikes without modifying the request pattern or batching behavior

62.A news aggregation platform wants to augment an Amazon Bedrock powered foundation model with a knowledge base containing more than 80 million articles stored in Amazon S3. The engineering team needs to generate embeddings for all historical content using Amazon Titan Text Embeddings and must continuously update embeddings for tens of thousands of new articles arriving daily. The solution must scale to large batch workloads, recover gracefully from failures, control costs, and avoid API throttling or timeouts during peak ingestion. What do you recommend to architect this large scale embedding workflow? (Select two)

1. Load all article text into a single Amazon Bedrock Knowledge Base and allow Bedrock to re-embed the full dataset on a scheduled basis to keep embeddings up to date for both historical and new content
2. Use an event driven pipeline with Amazon SQS and AWS Lambda to generate embeddings for new articles as they arrive and scale automatically
3. Create a Step Functions state machine that reads all article content, generates embeddings sequentially, and stores them in a vector database. Then invoke the same workflow for new articles by triggering the state machine on every S3 event
4. Use AWS Batch with array jobs to process historical documents stored in S3 and invoke Amazon Bedrock Titan embeddings in parallel while relying on Batch retry strategies for resilience
5. Use Amazon EventBridge to trigger Glue jobs for each new article so the pipeline continuously updates embeddings without manual intervention

63.A healthcare startup trains and retrains generative models on sensitive clinical notes and imaging metadata across multiple environments in different AWS accounts. Auditors now require a complete lineage graph that shows exactly which datasets, preprocessing jobs, training jobs, and model artifacts contributed to each production endpoint, including associations across accounts used for experimentation. How would you architect a lineage tracking solution that creates and connects artifacts, actions, and contexts so that auditors can reconstruct every model's history?

1. Use Amazon SageMaker ML Lineage Tracking to programmatically create artifacts for datasets, preprocessing jobs, training jobs, and model packages, and associate them through actions and contexts into a directed acyclic graph. Integrate the AddAssociation API with cross account IAM roles so that every experiment account records lineage back to the central production account endpoints
2. Configure AWS Glue crawlers to catalog all clinical and imaging datasets in the AWS Glue Data Catalog and rely on table relationships and job bookmarks to infer lineage across ETL and training workflows. Use Athena queries over the Data Catalog to reconstruct which datasets and jobs contributed to each model when auditors request a lineage report
3. Enable Amazon CloudWatch Logs and AWS CloudTrail for all training and endpoint invocations and export the logs into Amazon S3, then build custom Athena views that join log events to approximate lineage across preprocessing and training jobs. Rely on log timestamps and resource names to manually reconstruct model histories during audits
4. Use Amazon SageMaker ML Lineage Tracking to record training jobs and model packages as artifacts in the production account, and capture dataset and preprocessing job information in tags and CloudWatch Logs. Use a separate reporting script that combines lineage APIs, tags, and logs to build lineage graphs when needed

64.A bank wants to enforce version control, approval workflows, rollback capability, and per environment foundation model configuration variations for its GenAI compliance systems. The compliance team must compare historical model configurations against newly proposed versions before promoting them to production. The bank also requires a fully auditable workflow that tracks who approved each configuration change and which version is active in each environment. Which of the following solutions best support these requirements with minimum overhead?

1. Use Amazon Bedrock Prompt Management to store, version, and approve prompt templates, while allowing compliance reviewers to compare historical prompt versions with new ones. Leverage Prompt Management to implement controlled approval workflows and promote prompt configurations across development, staging, and production environments
2. Use a DynamoDB based configuration registry where each model configuration is stored as a separate item with a version number and environment label. Manually update environment specific configuration rows and archive older versions for comparison when evaluating promotion readiness
3. Use an internal Git based workflow to store configuration JSON files and prompt templates, combined with CodePipeline to deploy updates to Lambda functions that wrap model calls. Leverage Git tags and pull requests as approval checkpoints and rely on environment specific branches to manage configuration drift
4. Use Amazon Bedrock Prompt Router to route persona specific or compliance specific requests to predefined model configurations. Define routing rules and use routing metadata to control and track which configurations different environments use

65.A customer support application intermittently returns generic "model invocation failed" errors when sending structured JSON prompts to a foundation model API. Initial checks reveal that some requests include optional fields in unexpected formats and unescaped characters that were not present during earlier testing. How should you design request validation and error handling so that malformed inputs are detected early and failures are easier to diagnose?

1. Configure Amazon API Gateway to perform basic request passthrough and add a Lambda function that escapes special characters before invoking the foundation model API. Log the preprocessing results in CloudWatch Logs to help identify faulty payloads during troubleshooting
2. Implement schema validation in Amazon API Gateway using request models to enforce strict JSON structure and run a Lambda preprocessor that sanitizes and escapes user supplied fields before calling the foundation model. Capture detailed validation failures and preprocessing errors in Amazon CloudWatch Logs to provide clear diagnostics for debugging issues
3. Send all requests directly to the model endpoint and customize the foundation model itself to reject malformed fields. Enable CloudWatch metric filters to alert developers whenever the model responds with an invocation error
4. Implement an Amazon SQS buffering layer that queues all incoming requests and use an Amazon Lambda consumer to normalize field names before invoking the model API. Depend on SQS dead letter queues to capture malformed payloads that cannot be processed successfully